Нейросети для игр

Нейросети для игр

Проблема

В разделе про ИИ врагов мы вручную запрограммировали поведение: если игрок близко - атакуй, если далеко - патрулируй. В генетических алгоритмах мы автоматически подобрали *параметры* (скорость, HP, урон), но правила по-прежнему писали руками. И в автомате, и в ГА есть одна общая проблема: человек должен заранее знать, **какие правила нужны**.

А что, если мы не знаем правил? Что, если вместо "если HP < 30% - беги" мы хотим, чтобы враг *сам разобрался*, когда бежать?

Наивный подход: гигантская таблица, где для каждой комбинации входов записано действие.

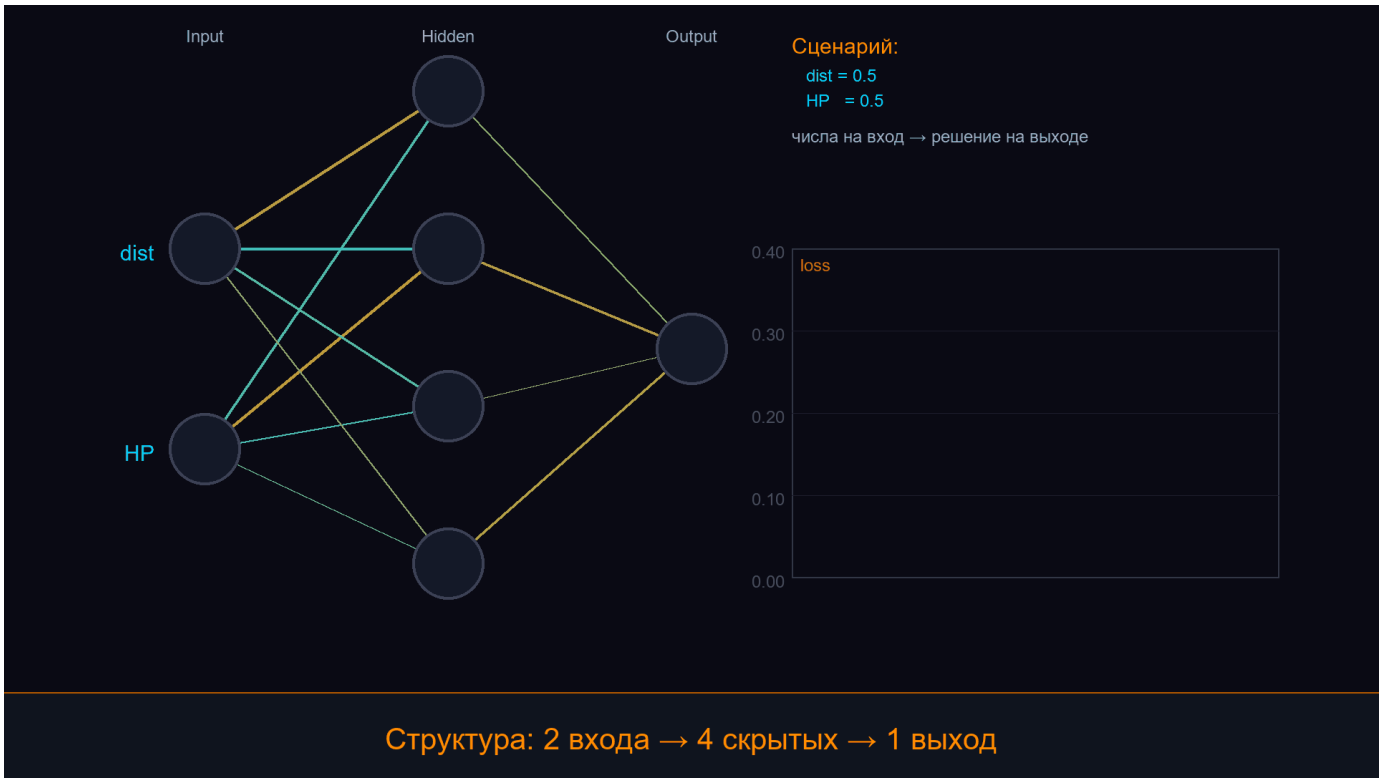
Python C#

```python
# lookup table: (hp_bucket, distance_bucket) -> action
ai_table = {
    ("high", "far"):   "patrol",
    ("high", "mid"):   "chase",
    ("high", "close"): "attack",
    ("low",  "far"):   "patrol",
    ("low",  "mid"):   "flee",
    ("low",  "close"): "flee",
}

def decide(hp_ratio, distance):
    hp_bucket = "low" if hp_ratio < 0.3 else "high"
    dist_bucket = "close" if distance < 50 else "mid" if distance < 150 el
    return ai_table.get((hp_bucket, dist_bucket), "patrol")
```

Проблема: добавьте ещё один вход (количество врагов рядом, наличие укрытия, кулдаун способности) - таблица растёт экспоненциально. 5 входов по 3 варианта - уже 243 строки. 10 входов - 59 049. И каждую строку нужно заполнить вручную.

А что, если вместо таблицы - **функция**, которая *сама подстраивается* под данные?



Нейрон: взвешенная сумма + активация

Название "нейрон" - отсылка к биологии. Настоящий нейрон в мозге получает сигналы от других нейронов через **синапсы**, какие-то сигналы усиливает, какие-то ослабляет, и если суммарное возбуждение превышает порог - "выстреливает" сигнал дальше. Математическая модель делает то же самое, только грубее.

Один искусственный нейрон - это простейший "решатель". Он берёт несколько входов, умножает каждый на свой **вес** (аналог силы синапса), складывает, добавляет **смещение** (bias - аналог порога) и пропускает через **функцию активации** (аналог "выстрелил / не выстрелил"):

$$\text{output} = \text{activation}(x_1 \cdot w_1 + x_2 \cdot w_2 + \ldots + b)$$

Что тут что:

- **Вес** (weight) - "важность" входа. Большой положительный вес = "этот вход сильно влияет на решение". Отрицательный = "этот вход работает *против*". Близкий к нулю = "этот вход неважен".
- **Смещение** (bias) - порог срабатывания. Сдвигает точку, в которой нейрон "переключается" с одного решения на другое.

Допустим, у врага два входа: `distance` (0–1) и `hp_ratio` (0–1). Нейрон с весами `[-2.0, +1.5]` и bias `0.3` "думает": чем ближе игрок (distance маленький) и чем больше HP, тем выше выход. Отрицательный вес у distance означает, что малая дистанция *увеличивает* взвешенную сумму (потому что `-2.0 × 0.1 = -0.2`, а `-2.0 × 0.9 = -1.8`).

Но взвешенная сумма - это любое число от -∞ до +∞. Нам нужно сжать его в удобный диапазон. Для этого - **функция активации**:

- **Sigmoid**: `σ(x) = 1 / (1 + e^(-x))` - гладкая S-кривая, сжимает любое число в диапазон (0, 1). Удобно интерпретировать как "уверенность": 0.9 = почти наверняка атаковать, 0.1 = почти

наверняка бежать.

- **ReLU**: `max(0, x)` - проще, быстрее, используется в современных глубоких сетях. Всё отрицательное обнуляется, положительное проходит как есть.

Python C#

```python
def neuron_forward(inputs, weights, bias):
    total = sum(w * x for w, x in zip(weights, inputs)) + bias
    return 1.0 / (1.0 + math.exp(-total))  # sigmoid
```

Веса и смещение инициализируются случайно в диапазоне `[-1, 1]`. Случайно - потому что мы пока не знаем, какие значения правильные. Обучение (о нём ниже) постепенно подгонит их под задачу. Если инициализировать всё нулями - нейроны будут одинаковыми и не смогут научиться разным вещам.

Грубо говоря, один нейрон - это прямая линия в пространстве входов: "всё по одну сторону - да, по другую - нет". Для нашего врага: "если `distance * (-2) + hp * 1.5 + 0.3 > 0` - атакуй". Прямая линия разделяет два решения. Но что, если граница между "бежать" и "атаковать" не прямая?

Нейрон - это то же самое, что **взвешенный выбор** из раздела про случайность, только веса подстраиваются автоматически. Там мы вручную задавали вероятности дропа, а здесь сеть сама учится, какие входы важнее.

Многослойная сеть (MLP)

Решение: **слои**. Несколько нейронов в первом слое проводят каждый свою прямую. Второй слой комбинирует их результаты - и получает уже кривую границу. Третий слой может скомбинировать кривые во что-то ещё сложнее. Входной слой принимает данные, скрытые слои преобразуют их, выходной выдаёт ответ. Это **многослойный перцептрон** (MLP - multilayer perceptron).

Данные для сети `[2, 4, 1]` (2 входа, 4 скрытых, 1 выход): массив матриц весов (2x4, 4x1) и массив векторов смещений. Прямой проход - цикл по слоям:

```
для каждого слоя l:
    для каждого нейрона j в слое l:
        total = bias[l][j]
        для каждого входа k:
            total += weights[l][j][k] * current[k]
        next[j] = sigmoid(total)
    current = next
return current
```

Три вложенных цикла - по слоям, по нейронам, по входам. Это и есть весь forward pass. Для сети `[2, 4, 1]` - это 2×4 + 4×1 = 12 умножений и сложений. Никакой магии. Магия - в том, как подобрать веса, чтобы выход был полезным.

> **Теорема об универсальной аппроксимации**: достаточно большая сеть с одним скрытым слоем может приблизить *любую* непрерывную функцию. Т.е. нейросеть - это универсальный аппроксиматор. Вопрос - как найти правильные веса.

Обучение: как сеть подстраивается

Ок, идея обучения:

1. **Прямой проход** - подаём данные, получаем выход.
2. **Ошибка (loss)** - сравниваем выход с правильным ответом. Разница - это ошибка.
3. **Обратное распространение** - вычисляем, какой вклад в ошибку внёс каждый вес.
4. **Корректировка** - сдвигаем веса в сторону уменьшения ошибки.

Интуиция: если ответ слишком большой - уменьши веса, которые к этому привели. Если слишком маленький - увеличь.

Градиентный спуск - это тот самый механизм корректировки. Представьте, что ошибка - это холмистый ландшафт. Вы стоите на склоне и хотите спуститься в самую низкую точку. Градиент - это направление самого крутого подъёма. Шагаем *против* градиента - и ошибка уменьшается. Стохастический градиентный спуск (SGD) - это применение этой идеи не ко всем данным сразу, а к случайным мини-батчам.

Ключевые формулы

Loss (ошибка) - среднеквадратичная разница между выходом и целью:

```
loss = (output - target)²
```

Дельта выходного слоя - насколько нужно скорректировать:

```
delta_output = (output - target) * sigmoid_deriv(output)
// sigmoid_deriv(x) = x * (1 - x)  - convenient, computed from the output itse
```

Дельта скрытого слоя - ошибка "проталкивается" назад через веса:

```
delta_hidden[j] = (сумма: weight[k][j] * delta_next[k]) * sigmoid_deriv(activa
```

Обновление весов - сдвигаем в сторону уменьшения ошибки:

```
weight[j][k] -= learning_rate * delta[j] * activation_prev[k]
bias[j]     -= learning_rate * delta[j]
```

Всё. Четыре формулы - это и есть backpropagation. Цикл обучения тогда выглядит как-то так: forward pass -> вычислить loss -> backward (дельты от выхода к входу) -> обновить веса -> повторить N эпох. Для операции XOR (классический тест для подобных штук) сеть `[2, 4, 1]` сходится за ~200 эпох.

Пример: сеть учится управлять врагом

Соединим всё вместе. У врага два входа: `distance_to_player` (0–1, нормализованная) и `hp_ratio` (0–1). Выход - одно число: 0 = бежать, 0.5 = патрулировать, 1 = атаковать.

Шаг 1. Генерируем обучающие данные из нашего ручного автомата. 200 случайных пар `(distance, hp_ratio)` -> прогоняем через FSM -> получаем целевое значение (0 = flee, 0.5 = patrol, 0.75 = chase, 1.0 = attack).

Шаг 2. Обучаем сеть `[2, 4, 1]` на этих парах, 300 эпох.

Шаг 3. Проверяем:

```
dist=0.1  hp=0.8 ->  ~1.0   (attack)  ✓
dist=0.5  hp=0.5 ->  ~0.7   (chase)   ✓
dist=0.8  hp=0.2 ->  ~0.1   (flee)    ✓
dist=0.45 hp=0.35 ->  ~0.55  (???)     - промежуточное значение!
```

Сеть воспроизвела логику автомата, **не зная правил** - только примеры. Более того, на границах между "атаковать" и "патрулировать" сеть выдаёт плавные промежуточные значения, а не резкий скачок. Это может создавать более естественное поведение.

Обучение с подкреплением (RL) - учиться на опыте

Допустим, у нас есть данные с правильными ответами - это **supervised learning**, и мы его только что разобрали. Но откуда взять "правильные ответы" для поведения врага в новой игре?

Reinforcement learning (RL) - другой подход. У нас нет ответов, есть только **награда**:

1. **Агент** (враг) наблюдает **состояние** (HP, расстояние, окружение).
2. Выбирает **действие** (атака, бег, патруль).
3. Получает **награду** (+10 за попадание, -50 за смерть, -1 за бездействие).
4. Корректирует стратегию, чтобы максимизировать суммарную награду.

Это цикл **Agent -> Action -> Environment -> Reward -> Agent**. Агент не знает правил - он пробует, ошибается, получает обратную связь и постепенно учится.

Q-learning: для каждой пары (состояние, действие) запоминаем ожидаемую награду. Со временем таблица Q-значений сходится к оптимальной стратегии. Но таблица огромна - и вот снова та же проблема: заменяем таблицу нейросетью. Это **Deep Q-Network (DQN)** - архитектура, которая победила человека в играх Atari.

Связь с другими подходами:

- Минимакс перебирает дерево игры. RL *обучается* оценивать позиции без полного перебора.
- Генетический алгоритм эволюционирует параметры через популяцию. RL оптимизирует *стратегию* одного агента через опыт.

Нейроэволюция: ГА + нейросеть

Хм, backprop требует дифференцируемой функции потерь - нужно вычислить градиент. Но что, если градиент посчитать нельзя? Например, фитнес врага - это результат целой симуляции боя, которая не дифференцируема.

Решение: **нейроэволюция**. Хромосома генетического алгоритма - это развёрнутые в список веса нейросети. Фитнес - производительность сети в игре. Скрещивание и мутация - на весах.

NEAT (NeuroEvolution of Augmenting Topologies) идёт ещё дальше: эволюционирует не только веса, но и **структуру** сети - какие нейроны существуют, как соединены. Начинает с минимальной сети и постепенно усложняет её.

Когда использовать нейроэволюцию вместо backprop:

- Фитнес не дифференцируем (результат симуляции).
- Нужно найти не только веса, но и архитектуру.
- Простая реализация - не нужен фреймворк для автоградиента.

Одна архитектура - разные задачи

Знакомый паттерн: тот же код, разные данные - разный результат.

Задача	Входы	Выходы	Обучение
ИИ врага	HP, расстояние, состояние	Действие (0–1)	Supervised / RL
Оценка позиции	Доска (сетка)	Оценка [−1, +1]	Supervised (AlphaZero)
Генерация уровней	Шум / seed	Сетка тайлов	GAN / RL

Задача	Входы	Выходы	Обучение
Адаптивная сложность	Статистика игрока	Множитель сложности	Online learning
Распознавание жестов	Координаты точек	Класс жеста	Supervised

Архитектура одна. Меняется то, что подаём на вход и чему учим. Как таблица параметров врагов из ИИ врагов - один автомат, разные числа.

Когда НЕ нужна нейросеть

Нейросеть - мощный инструмент. Но не в каждой бочке затычка.

- **Простое поведение** -> FSM или BT. Читаемо, отлаживаемо, предсказуемо.
- **Известные правила** -> кодируйте их напрямую. Зачем учить то, что вы уже знаете?
- **Маленькое пространство входов** -> таблица поиска. Быстрее и надёжнее.
- **Нужна объяснимость** -> нейросеть - чёрный ящик. Когда босс внезапно убегает, а вы не можете объяснить почему - это проблема.
- **Нет данных** -> без данных или среды для RL нейросеть не обучится.

Если задача решается `if-else` - нейросеть не нужна. Нейросеть - для задач, которые крайне проблематично сформулировать в виде правил.

Нейросети - огромная область, вот, что еще можно посмотреть:

- **CNN (свёрточные сети)** - для изображений и игровых сеток. Вместо полного соединения - маленькие фильтры, которые "скользят" по входу. Идеально для анализа тайловой карты или скриншота.
- **RNN / LSTM** - для последовательностей. Диалоги NPC, последовательности действий, музыкальная генерация. Сеть "помнит" предыдущие входы.
- **Трансформеры** - механизм внимания (attention). Основа современных LLM. В играх - для генерации текста, квестов, диалогов.
- **AlphaZero = MCTS + нейросеть** - нейросеть оценивает позицию и предлагает ходы, MCTS (Monte Carlo Tree Search) уточняет дерево поиска. Побеждает людей в шахматах, Go, Shogi. Связь: минимакс -> MCTS -> AlphaZero.
- **GAN (генеративно-состязательные сети)** - две сети соревнуются: генератор создаёт контент, дискриминатор пытается отличить от настоящего. Применение: генерация текстур, уровней, спрайтов.
- **Multi-agent RL** - несколько агентов обучаются одновременно, играя друг против друга. Создаёт сложные стратегии без ручного программирования.

Итого

Концепция	Что делает
Нейрон	Взвешенная сумма входов + активация -> одно число
Слой	Группа нейронов, обрабатывающих данные параллельно
Прямой проход	Подать данные на вход, получить ответ на выходе
Функция потерь	Мера ошибки: насколько ответ далёк от правильного
Обратное распространение	Вычислить вклад каждого веса в ошибку
Градиентный спуск	Сдвинуть веса в сторону уменьшения ошибки
RL	Обучение через награду от среды, без правильных ответов
NEAT	Эволюция весов и структуры нейросети

Что	Где подробнее
Ручной ИИ врагов, который сеть может заменить	Простой ИИ врагов
Деревья поведений -> обучаемые деревья	Деревья поведений
Оценочная функция, которую сеть выучивает	Минимакс
Эволюция весов и топологии (NEAT)	Генетические алгоритмы
SGD, случайные подвыборки, exploration	Случайность и вероятность
Генерация как обучающая среда	Процедурная генерация

Powered by Wiki.js